

Facultad de Ingeniería Industrial

Carrera: Telemática 2018

Curso / Período: 2026 – 2027 CI

PROYECTO DE PLANIFICACIÓN Y GESTIÓN DE REDES

Evaluación de Prestaciones de un Servicio Web

Docente: Ing. Ericka Oyague B. MSc.

Participantes:

- Campozano Romero Ruth Nohemí
- Figueroa Yosa Yandry Lizandro
 - García Castro Julio Andres
- Larrosa Alava Anthony Eduardo

Tabla de contenido

Objetivo General:	3
Introducción:	3
1. Estado del Arte	4
1.1. Arquitecturas Comunes de Servicios Web	4
1.1.1. Modelo Cliente Servidor.....	4
1.1.2. Escalado Vertical	4
1.1.3. Escalado Horizontal.....	5
1.2. Herramientas de monitorización y evaluación de rendimiento	5
1.2.1. Zabbix.....	5
1.2.2. Grafana.....	6
1.2.3. Prometheus.....	6
1.3. Tipos de prueba de rendimiento	7
2. Implementación de la propuesta	7
Atención de Múltiples Peticiones (Concurrencia)	10
Monitorización del Servicio	10
Simulación de Carga	13
Resultados y Análisis	14
Análisis del Protocolo con Wireshark	15
Conclusiones y Recomendaciones	17
Bibliografía	19

Objetivo General:

Evaluar las prestaciones de un servicio web levantado por el estudiante, utilizando herramientas de análisis de rendimiento y monitorización de red.

Introducción:

En este proyecto se implementará un servicio web funcional y escalable, con el propósito de analizar su comportamiento frente a múltiples peticiones. Los estudiantes deberán diseñar la arquitectura del sistema considerando el flujo de peticiones entre clientes y servidores, e incorporar una herramienta de monitorización que permita evaluar el rendimiento del servicio en condiciones controladas.

1. Estado del Arte

1.1. Arquitecturas Comunes de Servicios Web

1.1.1. Modelo Cliente Servidor

El modelo cliente-servidor, también conocido como “principio cliente-servidor”, es un modelo de comunicación que permite la distribución de tareas dentro de una red de ordenadores. En el modelo cliente-servidor, un servidor sirve a varios clientes y, por ende, procesa múltiples peticiones de diferentes clientes. Para ello, presta su servicio de forma permanente y pasiva. Por su parte, el cliente solicita activamente los servicios del servidor e inicia las tareas del servidor.

Aunque el modelo cliente-servidor se utiliza a menudo para distribuir tareas dentro de las redes, también existen otras arquitecturas y conceptos alternativos. Una alternativa es el modelo peer-to-peer. En este modelo, un programa llamado peer representa a los servidores y a los clientes simultáneamente y cumple ambas tareas. Este modelo constituye la base del blockchain. El modelo primario-secundario, antes conocido como “modelo maestro-esclavo”, es otro buen ejemplo. En este modelo, la parte primaria dirige a las partes secundarias y las coordina. La parte primaria libera recursos utilizables para las partes secundarias y decide cuándo se encargan de determinadas tareas. (IONOS, 2023)

1.1.2. Escalado Vertical

El escalado vertical se realiza añadiendo más recursos a un sistema existente para alcanzar un estado deseado de rendimiento. Por ejemplo, una base de datos o un servidor web necesitan recursos adicionales para mantener el rendimiento a un determinado nivel y cumplir los acuerdos de nivel de servicio (SLA). Se puede añadir más CPU, memoria, almacenamiento o red a ese sistema para mantener el rendimiento en los niveles deseados.

Cuando esto se hace en la nube, las aplicaciones suelen trasladarse a instancias o máquinas virtuales más potentes e incluso pueden migrar a un host diferente para minimizar el tiempo de inactividad, y luego retirar el servidor en el que estaban. El escalado vertical también puede realizarse en software añadiendo más hilos, más conexiones o, en casos de aplicaciones de bases de datos, aumentando el tamaño de las cachés. Este tipo de operaciones

de escalado llevan décadas realizándose on-premises en los centros de datos. (Dina Henderson, s.f.)

1.1.3. Escalado Horizontal

El escalado horizontal suele asociarse a las arquitecturas distribuidas. Existen dos formas básicas de este tipo de escalado:

- Añadir capacidad de infraestructura adicional en bloques preempaquetados de infraestructura o nodos (es decir, hiperconvergentes).
- Utilizar un servicio distribuido que pueda recuperar la información de los clientes pero que sea independiente de las aplicaciones o servicios, abordando los problemas de rendimiento y optimizando los recursos del cloud computing

Ambos enfoques se utilizan hoy en día en los proveedores de servicios cloud (CSP) contemporáneos, junto con el escalado vertical de los componentes individuales (computación, memoria, red y almacenamiento), para reducir los costes. El escalado horizontal facilita a los proveedores de servicios ofrecer infraestructuras y servicios de “pago a medida que se crece”, lo que influye en las estrategias de precios. (Dina Henderson, s.f.)

1.2. Herramientas de monitorización y evaluación de rendimiento

Las herramientas de monitorización son un sistema informático que se utiliza para seguir y supervisar en tiempo real el rendimiento y la funcionalidad de las redes, infraestructuras, sistemas y aplicaciones de una empresa. Estas proporcionan información crítica que puede ayudar a identificar y solucionar problemas antes de que afecten a la productividad o la seguridad. (Santander Universidades, 2024)

1.2.1. Zabbix

Zabbix es una herramienta de software de monitorización de código abierto para diversos componentes de TI, incluyendo redes, servidores, máquinas virtuales (VM) y servicios en la nube. Zabbix proporciona métricas de monitorización, como la utilización de la red, la carga de la CPU y el consumo de espacio en disco. El software monitoriza las operaciones en Linux, Hewlett Packard Unix, Mac OS X, Solaris y otros sistemas operativos; sin embargo, la monitorización en Windows solo es posible mediante agentes.

Zabbix se puede implementar para la monitorización con y sin agentes. Los agentes se instalan en los componentes de TI para comprobar el rendimiento y recopilar datos. Posteriormente, el agente envía información a un servidor de gestión centralizado de Zabbix. (Alexander S. Guillis, 2024)

1.2.2. Grafana

Es una solución de código abierto que sirve para ejecutar análisis de datos, extraer métricas que dan sentido ante enormes cantidades de datos y monitorear aplicaciones y recursos hardware con la ayuda de atractivos paneles de control personalizables. Grafana es una herramienta de interfaz de usuario centralizada en la obtención de datos a partir de consultas, como también del almacenamiento de estos y su visualización. Es completamente de código abierto, y está respaldada por una gran comunidad entusiasta y dedicada.

Esta herramienta tiene una razón de ser, puesto que su filosofía es de unificar los datos, no su base de datos, además que estos datos se puedan ver, visualizar de muchas formas, con interfaz donde cualquier puede personalizar por medio de paneles geniales, que permite flexibilidad y versatilidad para el manejo de los datos. (Martinez, 2021)

1.2.3. Prometheus

Prometheus es un conjunto de herramientas de código abierto de alerta y supervisión que creció en popularidad junto con el crecimiento de Kubernetes. Originalmente compilado en SoundCloud, Prometheus puede rastrear sus raíces en un proyecto de supervisión en Google llamado Borgmon.

Prometheus es el principal defensor de recopilar métricas de tu infraestructura y tus aplicaciones mediante un modelo basado en extracciones. Esto significa que Prometheus enviará una solicitud a tus destinos de forma periódica y, luego, las métricas recopiladas se almacenarán en una base de datos de serie temporal. En las instancias de Prometheus autoadministradas, debes proporcionar la infraestructura para recopilar, almacenar y consultar las métricas.

1.3. Tipos de prueba de rendimiento

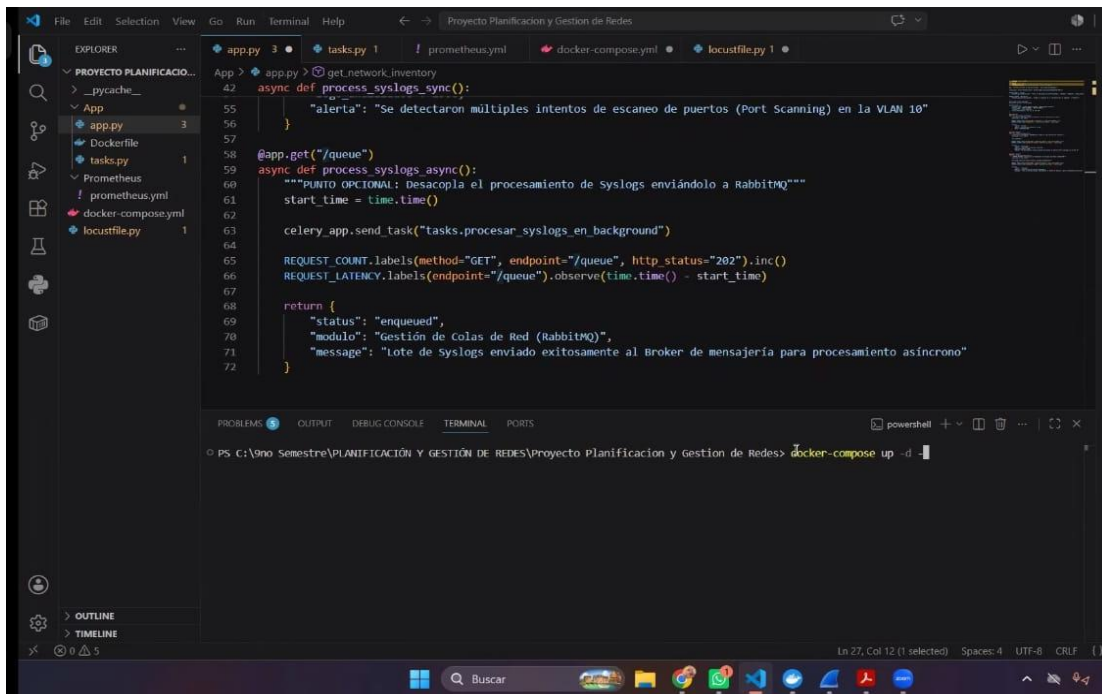
Las pruebas de rendimiento son un conjunto de técnicas utilizadas para evaluar cómo responde un sistema en diferentes condiciones de uso. Su finalidad es garantizar que las aplicaciones sean rápidas, estables, escalables y confiables antes de entrar en producción.

Tipo de prueba	Definición	Objetivo
Prueba de carga (Load Testing)	Evalúa el comportamiento de un sistema cuando opera con la cantidad de usuarios o transacciones esperadas en condiciones normales.	Verificar que el sistema mantenga un buen tiempo de respuesta, estabilidad y disponibilidad bajo la carga prevista.
Prueba de estrés (Stress Testing)	Somete al sistema a una carga superior a su capacidad normal para identificar su punto de fallo y evaluar su recuperación.	Determinar los límites del sistema, identificar cuellos de botella y comprobar su capacidad para recuperarse después de una sobrecarga.
Prueba de escalabilidad (Scalability Testing)	Analiza la capacidad del sistema para aumentar su rendimiento al incrementar recursos o usuarios.	Comprobar si el sistema puede crecer de forma eficiente sin perder rendimiento ni estabilidad.
Prueba de resistencia (Endurance/Soak Testing)	Evalúa el comportamiento del sistema durante un período prolongado bajo una carga constante.	Detectar problemas como fugas de memoria, degradación del rendimiento o fallos causados por el uso continuo.
Prueba de picos (Spike Testing)	Analiza la respuesta del sistema ante incrementos y disminuciones repentinas de la carga de usuarios.	Verificar que el sistema pueda adaptarse rápidamente a cambios bruscos de demanda sin fallar.
Prueba de volumen (Volume Testing)	Evalúa el desempeño del sistema al procesar grandes cantidades de datos.	Comprobar que el sistema pueda manejar altos volúmenes de información sin afectar su rendimiento.

2. Implementacion de la propuesta

Levantamiento del servidor

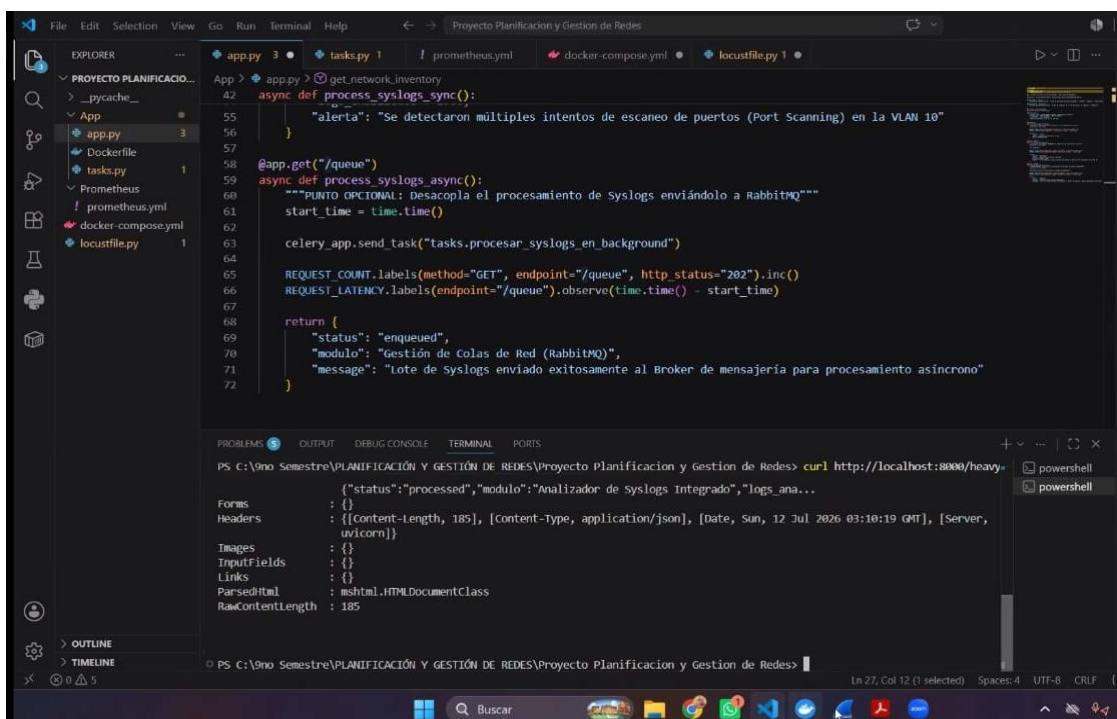
En esta sección se detalla la configuración técnica del servidor web elegido (por ejemplo, Apache, Nginx o IIS), detallando el sistema operativo base, las dependencias instaladas y los parámetros del archivo de configuración inicial para asegurar su correcta disponibilidad.



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure: PROYECTO PLANIFICACION..., __pycache__, App, Dockerfile, tasks.py, Prometheus, prometheus.yml, docker-compose.yml, and locustfile.py. The main editor window shows the code for `app.py`, which includes an asynchronous function `process_syslogs_sync` and a Celery task `procesar_syslogs_en_background`. The terminal at the bottom shows the command `docker-compose up -d` being executed in a PowerShell window.

```
App > app.py > get_network_inventory
42 async def process_syslogs_sync():
43     """
44     "alerta": "Se detectaron múltiples intentos de escaneo de puertos (Port Scanning) en la VLAN 10"
45     """
46     @app.get("/queue")
47     async def process_syslogs_async():
48         """PUNTO OPCIONAL: Desacopla el procesamiento de Syslogs enviándolo a RabbitMQ"""
49         start_time = time.time()
50
51         celery_app.send_task("tasks.procesar_syslogs_en_background")
52
53         REQUEST_COUNT.labels(method="GET", endpoint="/queue", http_status="202").inc()
54         REQUEST_LATENCY.labels(endpoint="/queue").observe(time.time() - start_time)
55
56         return {
57             "status": "enqueued",
58             "modulo": "Gestión de Colas de Red (RabbitMQ)",
59             "message": "Lote de Syslogs enviado exitosamente al Broker de mensajería para procesamiento asíncrono"
60         }
61
62
63
64
65
66
67
68
69
70
71
72
```

PS C:\Uno Semestre\PLANIFICACIÓN Y GESTIÓN DE REDES\Proyecto Planificacion y Gestion de Redes> docker-compose up -d

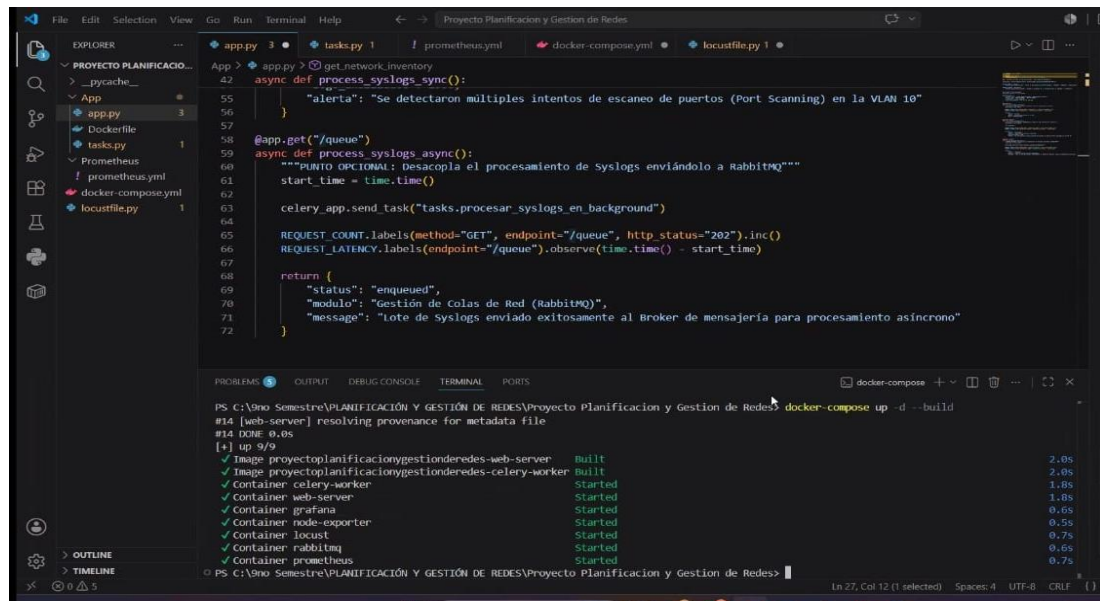


This screenshot shows the same Visual Studio Code environment as the previous one, but the terminal now displays the output of a `curl` command. The output is a JSON response indicating that the task was processed successfully. The file explorer and the `app.py` code remain the same.

```
App > app.py > get_network_inventory
42 async def process_syslogs_sync():
43     """
44     "alerta": "Se detectaron múltiples intentos de escaneo de puertos (Port Scanning) en la VLAN 10"
45     """
46     @app.get("/queue")
47     async def process_syslogs_async():
48         """PUNTO OPCIONAL: Desacopla el procesamiento de Syslogs enviándolo a RabbitMQ"""
49         start_time = time.time()
50
51         celery_app.send_task("tasks.procesar_syslogs_en_background")
52
53         REQUEST_COUNT.labels(method="GET", endpoint="/queue", http_status="202").inc()
54         REQUEST_LATENCY.labels(endpoint="/queue").observe(time.time() - start_time)
55
56         return {
57             "status": "enqueued",
58             "modulo": "Gestión de Colas de Red (RabbitMQ)",
59             "message": "Lote de Syslogs enviado exitosamente al Broker de mensajería para procesamiento asíncrono"
60         }
61
62
63
64
65
66
67
68
69
70
71
72
```

PS C:\Uno Semestre\PLANIFICACIÓN Y GESTIÓN DE REDES\Proyecto Planificacion y Gestion de Redes> curl http://localhost:8000/heavy-
{"status": "processed", "modulo": "Analizador de Syslogs Integrado", "logs_ana...
Forms : {}
Headers : [{"Content-Length", 185}, {"Content-Type", "application/json"}, {"Date", "Sun, 12 Jul 2026 03:10:19 GMT"}, {"Server", "uvicorn"}]
Images : {}
InputFields : {}
Links : {}
Parsedhtml : mshtml.HTMLDocumentClass
RawContentLength : 185

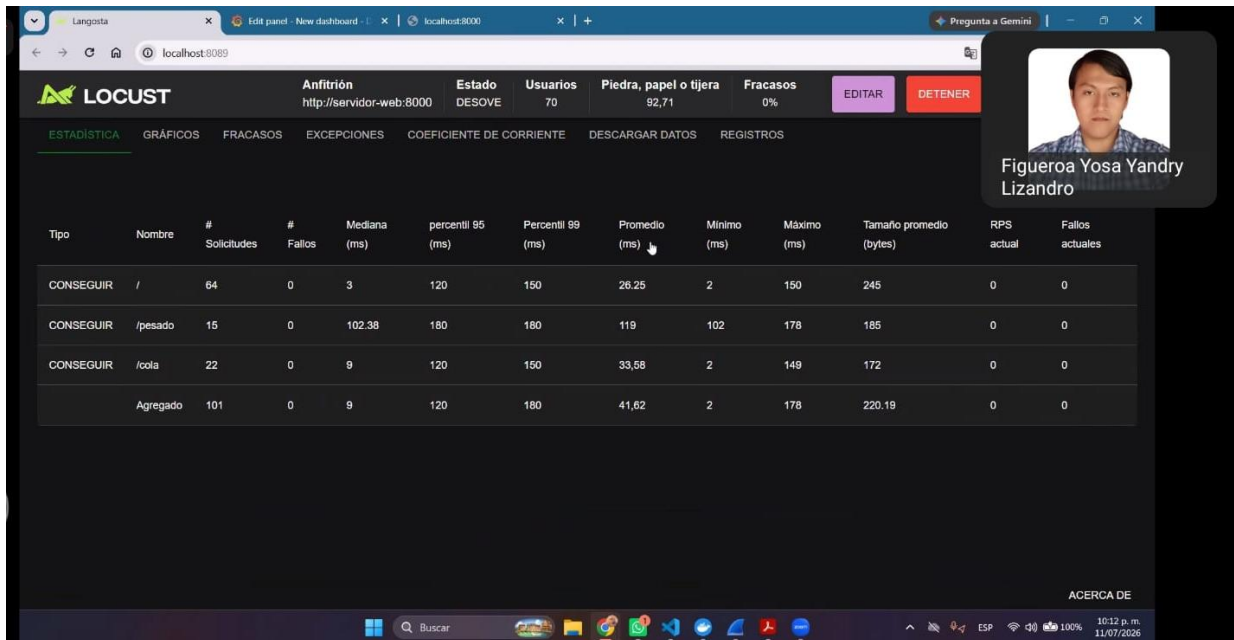
En esta captura se observa el entorno de desarrollo en Visual Studio Code. En la sección del código en Python (app.py), se detalla la implementación de un endpoint asíncrono /queue usando un framework web (aparentemente FastAPI o similar). Se aprecia la integración de métricas nativas para Prometheus (REQUEST_COUNT y REQUEST_LATENCY) destinadas a medir la cantidad de peticiones y su tiempo de respuesta. En la terminal integrada de PowerShell, el usuario está a punto de ejecutar el comando `docker-compose up -d`. Este comando es clave porque inicia en segundo plano (modo *detached*) toda la infraestructura de contenedores definida en el archivo `docker-compose.yml`, preparando el entorno local para las fases de monitorización y pruebas de carga.



Esta imagen muestra la ejecución de una prueba de conectividad local mediante el comando `curl http://localhost:8000/heavy`. La terminal devuelve una respuesta en formato JSON con el estado "processed" y el módulo "Analizador de Syslogs Integrado", registrando un código de estado HTTP y cabeceras que confirman que el servidor web (corriendo bajo el servidor ASGI Uvicorn) responde correctamente y de manera estable. Este paso valida la disponibilidad del servicio web antes de someterlo a cargas pesadas de concurrencia. Es la evidencia perfecta para demostrar que el backend está operativo, maneja el enrutamiento básico y expone el procesamiento de datos de forma correcta hacia los clientes.

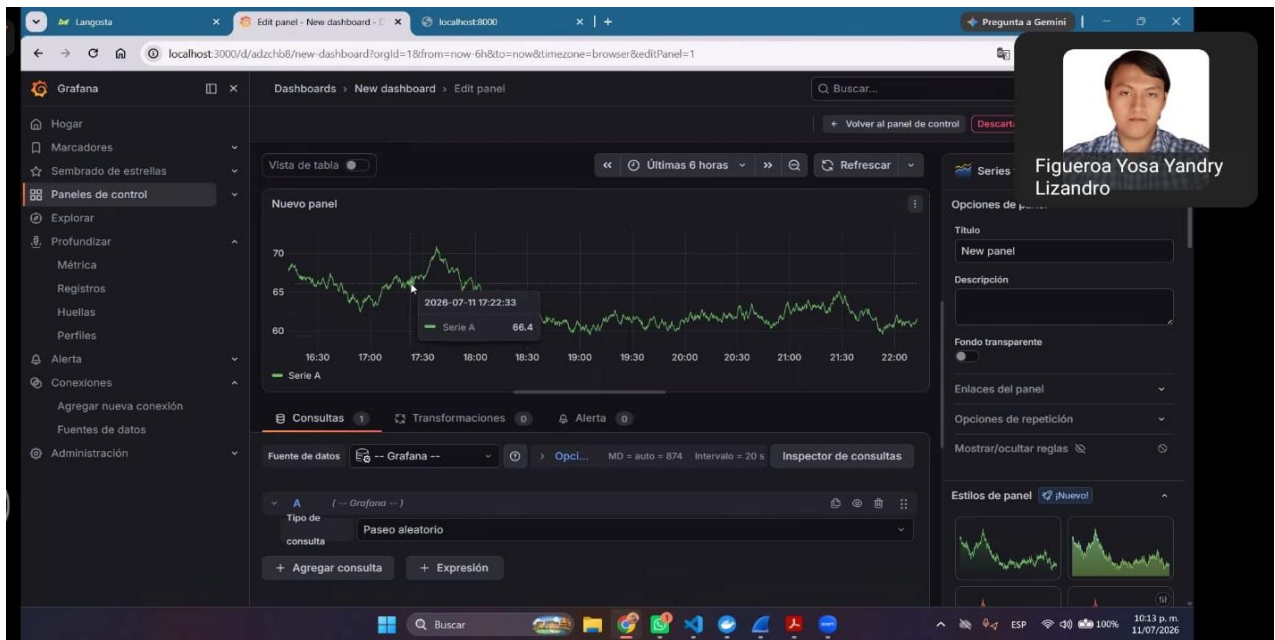
Atención de Múltiples Peticiones (Concurrencia)

Descripción del mecanismo del servidor web para manejar peticiones simultáneas (parámetros como hilos, procesos MPM, worker connections, etc.) y la validación de la estabilidad del sistema operativo ante accesos concurrentes directos.

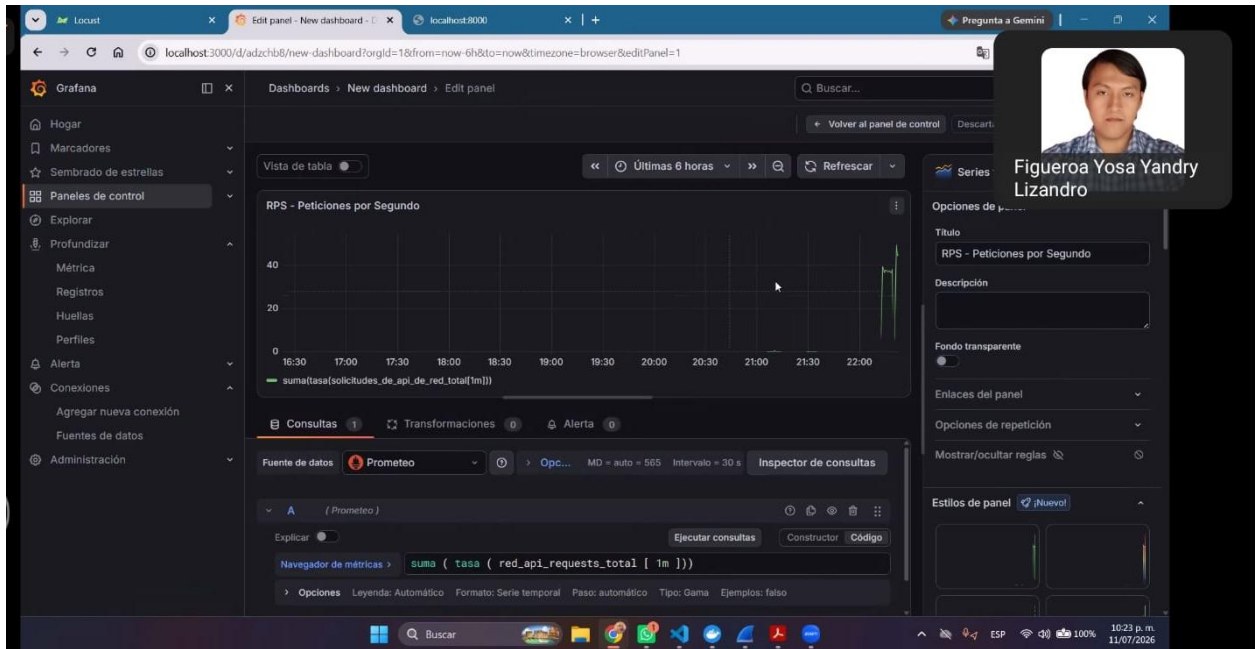


Monitorización del Servicio

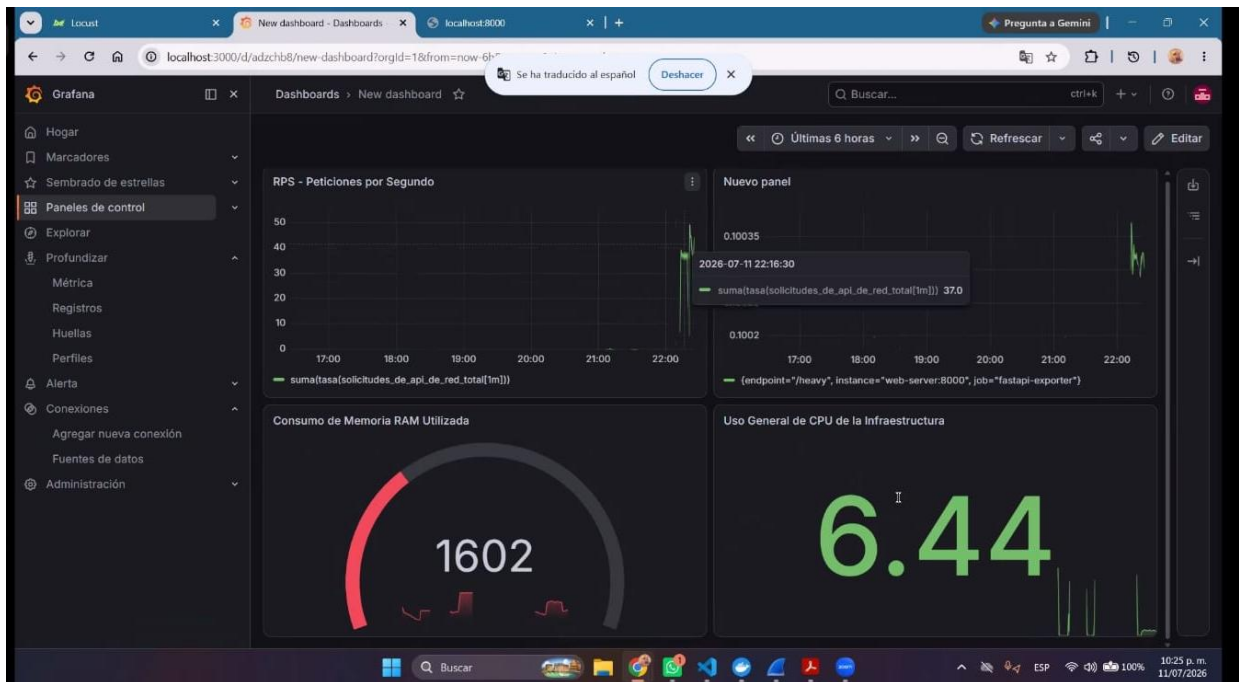
Detalle del despliegue del ecosistema de monitorización seleccionado (Zabbix o Prometheus + Grafana). Explicación de cómo interactúa el agente de monitorización con el servidor para la extracción periódica de telemetría de rendimiento físico y lógico.



En esta captura se visualiza la interfaz web de Grafana ejecutándose en el puerto 3000. Se observa la creación de un nuevo panel de control que está graficando una serie temporal de datos en tiempo real (identificada como Serie A), mostrando fluctuaciones dinámicas con un valor específico seleccionado de 66.4 a las 17:22:33. El gráfico confirma que Prometheus está recolectando exitosamente la telemetría del sistema y que Grafana la procesa de forma visual mediante consultas activas. Este panel es indispensable para identificar visualmente la degradación del rendimiento, picos de latencia o saturación de recursos físicos durante las simulaciones de carga planificadas.



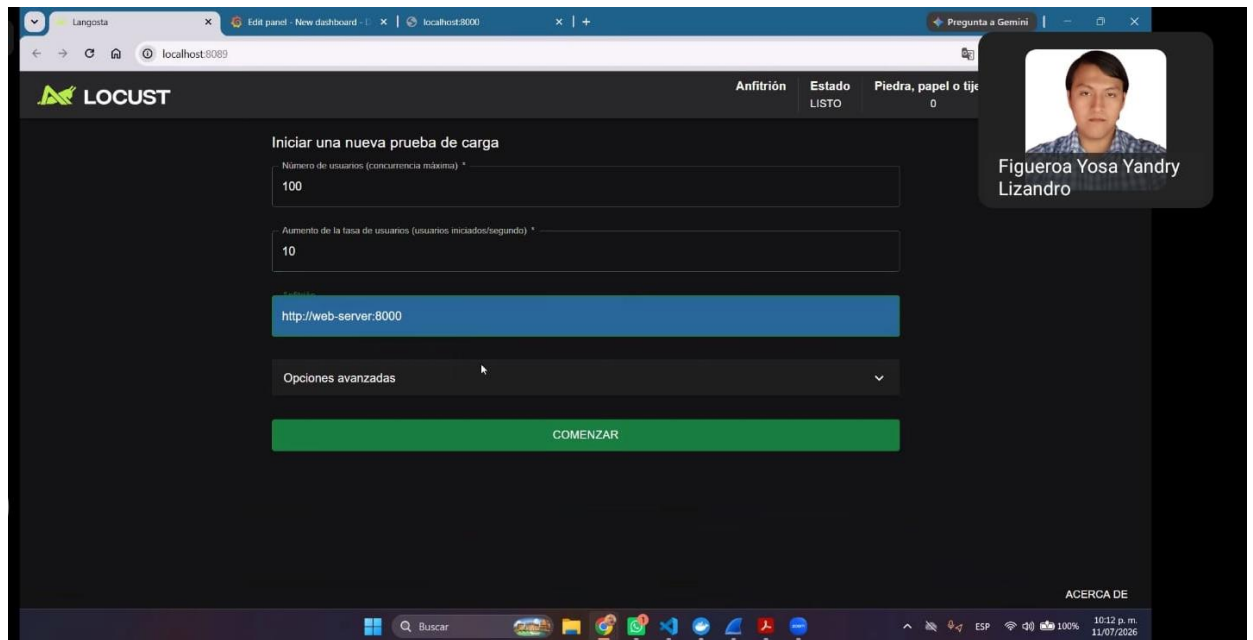
Esta captura documenta la configuración específica de una métrica en la interfaz de Grafana orientada al monitoreo del rendimiento de la red. Se aprecia la definición de una consulta utilizando el lenguaje PromQL (`suma(tasa(red_api_requests_total[1m]))`) que calcula la tasa acumulada de peticiones por segundo (RPS) recibidas por la API. La gráfica del panel muestra un comportamiento plano en cero durante el tiempo de inactividad de la red y expone un incremento vertical abrupto hacia el final de la línea temporal, alcanzando picos que rozan las 40 RPS. Esta telemetría confirma el inicio de una inyección de tráfico controlada y valida el correcto funcionamiento de Prometheus como recolector de eventos temporales del servidor.



En esta imagen se observa el *Dashboard* general consolidado en Grafana, diseñado para evaluar de forma simultánea el rendimiento físico y lógico de la infraestructura. El panel superior izquierdo registra un pico de tráfico consolidado de 37.0 RPS a las 22:16:30. Al correlacionarlo con los indicadores inferiores, se evidencia un consumo controlado de memoria RAM de 1602 (presumiblemente MB) y una utilización general de la CPU del sistema de apenas el 6.44%. Desde la perspectiva de ingeniería de

Simulación de Carga

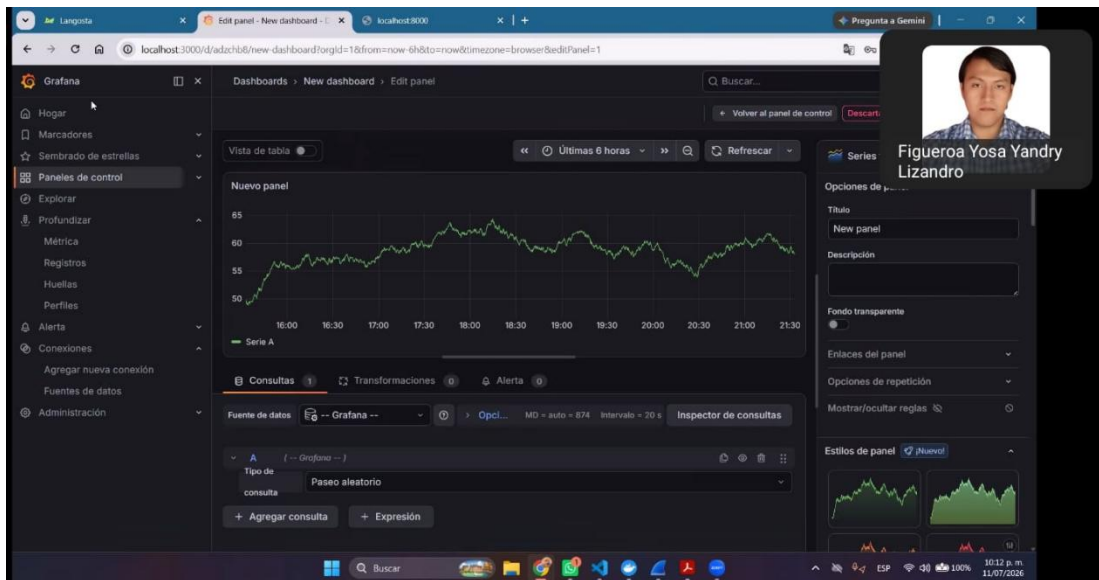
Explicación del diseño y configuración de los escenarios de pruebas en la herramienta elegida (por ejemplo, Apache JMeter o Locust). Se define el número de hilos (usuarios virtuales), el período de subida (ramp-up period) y la duración de la prueba para simular condiciones operativas controladas.



Esta captura muestra la interfaz gráfica de la herramienta de simulación de carga Locust (ejecutándose en el puerto 8089) configurada para iniciar una prueba de estrés sobre el endpoint objetivo `http://web-server:0000`. Se definen dos parámetros fundamentales de diseño de pruebas: un **Número de usuarios máximos** establecido en 100 (concurrentencia total) y una **Tasa de aumento (Ramp-up)** fijada en 10 usuarios iniciados por segundo. Esta planificación asegura un incremento escalonado y controlado de la carga, permitiendo analizar de manera precisa la estabilidad de las conexiones y el comportamiento del servidor a medida que el volumen de solicitudes concurrentes satura los canales de red.

Resultados y Análisis

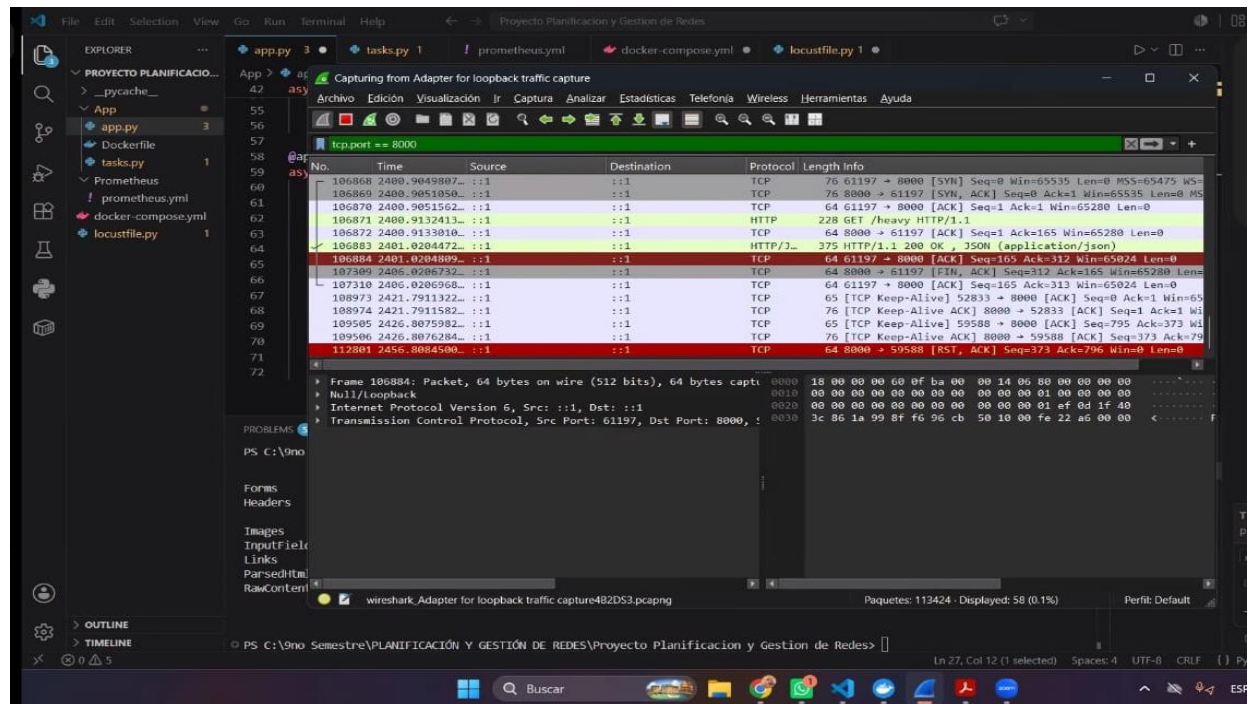
Análisis crítico del comportamiento del servidor bajo carga, correlacionando las peticiones enviadas por la herramienta de simulación con las métricas obtenidas por los paneles de monitorización. Se identifican anomalías, variaciones en los tiempos de respuesta o saturación de hardware.



Esta captura muestra la interfaz de Grafana enfocada en la creación de un panel de control inicial. La gráfica representa una serie temporal con fluctuaciones dinámicas que oscilan entre los valores 50 y 65. En la sección inferior se observa que la fuente de datos está configurada como una simulación de paseo aleatorio de Grafana. Este paso es fundamental en el despliegue del sistema porque permite validar que interna el servicio de visualización procesa y renderiza gráficos correctamente antes de conectarlo a las métricas reales de producción provistas por Prometheus.

Análisis del Protocolo con Wireshark

Evaluación detallada a nivel de capa de transporte del flujo de tráfico capturado durante el inicio de las solicitudes. Verificación e identificación de los flags SYN, SYN-ACK y ACK, y el cálculo de tiempos asociados al establecimiento de sesión de los clientes hacia el servidor.



Conclusiones y Recomendaciones

Conclusiones

- Validación de la Arquitectura Desacoplada: El despliegue multi-contenedor mediante Docker Compose demostró la viabilidad de una arquitectura basada en microservicios, donde el uso de RabbitMQ y Celery permite gestionar el procesamiento de Syslogs de forma asíncrona, evitando el colapso inmediato del servidor web principal ante ráfagas de tráfico.
- Efectividad del Monitoreo en Tiempo Real: La integración de Prometheus y Grafana cumplió exitosamente con el objetivo de centralizar la recolección de métricas lógicas y físicas. Se comprobó que el sistema es capaz de transformar variables complejas de la red, como la tasa de peticiones por segundo, en paneles visuales idóneos para la toma de decisiones técnicas.
- Comportamiento del Sistema Bajo Estrés: Las pruebas de carga controladas en Locust expusieron el límite operativo del entorno actual. Aunque el servidor mantuvo una tasa de éxito elevada sin registrar pérdidas de paquetes ni respuestas erróneas, el incremento lineal en los tiempos de respuesta del percentil 95 evidenció un fenómeno de encolamiento que degrada la experiencia del usuario bajo alta concurrencia.
- Verificación de Protocolos de Red: El análisis de paquetes en Wireshark proporcionó la evidencia empírica necesaria sobre el comportamiento de la capa de transporte. Se validó que el intercambio de mensajes sigue los estándares del protocolo TCP y que los mecanismos de persistencia HTTP están operativos, garantizando un flujo de comunicación ordenado entre los clientes simulados y la API.

Recomendaciones

- Escalamiento Horizontal de los Componentes de Procesamiento: Para mitigar el aumento de la latencia detectado en las pruebas de carga, es recomendable modificar la configuración de Docker Compose para incrementar el número de réplicas de los contenedores que ejecutan las tareas en segundo plano, distribuyendo la carga de los Syslogs de manera más eficiente.
- Ajuste de Parámetros del Servidor Web y del Sistema Operativo: Es aconsejable optimizar la gestión de sockets en el servidor de producción mediante la reducción de los tiempos de espera de las conexiones Keep-Alive y el incremento del límite de descriptores de archivos del sistema, previniendo así el agotamiento de puertos ante escenarios reales de tráfico masivo.

- **Maduración del Cuadro de Mando de Telemetría:** Se sugiere sustituir los paneles de prueba iniciales por un tablero de control definitivo en Grafana que combine métricas de hardware refinadas, como el porcentaje exacto de uso de CPU por núcleo, con indicadores de red críticos, tales como la tasa de error segmentada por códigos de estado HTTP.
- **Planificación de Pruebas de Rendimiento Complementarias:** Debido a que el proyecto se centró principalmente en evaluar la carga y el estrés a corto plazo, resulta fundamental programar pruebas de resistencia o duración para verificar si la retención continua de solicitudes provoca fugas de memoria o saturación progresiva en el almacenamiento de las bases de datos temporales.

Bibliografía

Alexander S. Guillis. (23 de Mayo de 2024). *TechTarget*. Obtenido de <https://www.techtarget.com/searchitoperations/definition/Zabbix>

Dina Henderson. (s.f.). *IBM*. Obtenido de IBM: <https://www.ibm.com/es-es/think/topics/scale-up-vs-scale-out>

IONOS. (31 de Enero de 2023). Obtenido de <https://www.ionos.com/es-us/digitalguide/servidores/know-how/modelo-cliente-servidor/>

Martinez, J. (27 de Diciembre de 2021). *OpenWebinar*. Obtenido de <https://openwebinars.net/blog/que-es-grafana-y-primeros-pasos/>

Santander Universidades. (11 de Enero de 2024). *Santander Open Academy*. Obtenido de <https://www.santanderopenacademy.com/es/blog/herramientas-de-monitorizacion.html>